ImgShare – Architektur & Verschlüsselung

Ende-zu-Ende-verschlüsselter Bild- und Text-Chat zwischen zwei Kollegen · Stand 07.11.2026 · Protokoll v1

1 · Überblick

ImgShare besteht aus zwei Teilen: einem Windows-Programm im Infobereich (Chatfenster, Bilder einfügen, Verlauf) und einem kleinen Relay-Server auf `https://server.example.com:47000`. Der Server ist absichtlich „dumm“: Er speichert und verteilt nur verschlüsselte Bytes. **Sämtliche Kryptografie passiert in den beiden Clients**; der Schlüssel verlässt nie die Rechner der Teilnehmer.

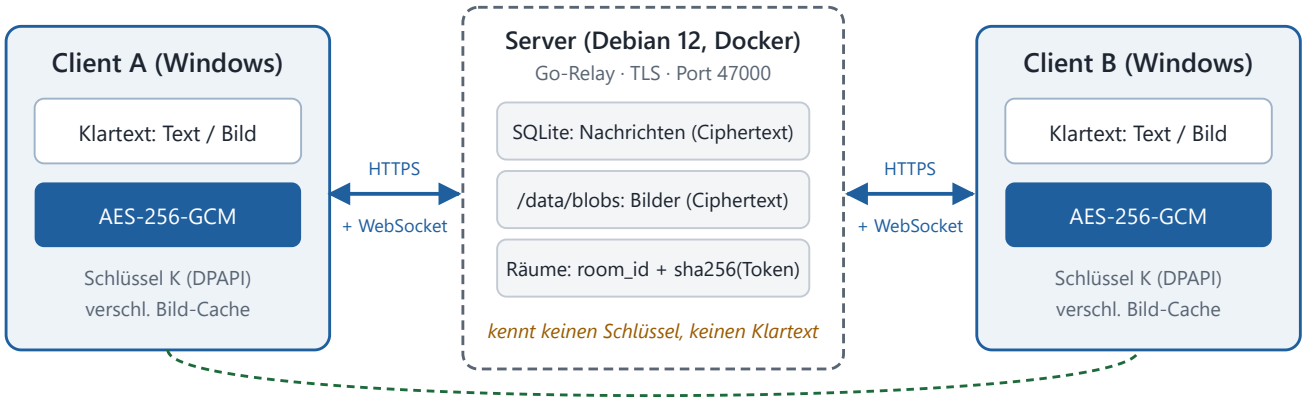


Abb. 1 – Komponenten und Vertrauensgrenzen. Alles, was den Client verlässt, ist bereits verschlüsselt; TLS kommt als zweite Schicht hinzu.

Komponente	Technik
Windows-Client	WPF auf .NET 10, Symbol im Infobereich, eigenständige Single-File-EXE (`ImgShare.exe`). Bibliothek `ImgShare.Core` enthält Krypto, API-Client und Konfiguration.
Server	Go 1.26, eine Binärdatei im Container `distroless/static` ; SQLite (pure Go) für Metadaten und Nachrichten, Dateisystem für Bild-Blobs.
Transport	REST über HTTPS zum Senden und Nachladen, WebSocket für Echtzeit-Zustellung und Online-Anzeige.
TLS-Zertifikat	Let's Encrypt vom Host, read-only gemountet; der Server lädt es nach einer Erneuerung automatisch neu.

2 · Schlüssel und Schlüsselableitung

Jeder Chat hat genau einen **Root-Key K**: 32 kryptografisch zufällige Bytes (256 Bit), erzeugt vom Client, der den Chat einrichtet. Er wird als Base32 in Vierergruppen angezeigt (z. B. `K7QF-2MXA-…`, Tippfehler-tolerant bei 0/O und 1/I) und an den Kollegen **außerhalb des Servers** weitergegeben – per QR-Code, persönlich oder über einen anderen sicheren Kanal.

Aus K werden per **HKDF-SHA256** (Salt `"imgshare-v1"`) unabhängige Teilschlüssel abgeleitet. Kennt man einen davon, lassen sich weder K noch die anderen berechnen:

Info	Länge	Ergebnis	Verwendung
`enc`	32 Byte	enc_key	AES-256-GCM-Schlüssel für Nachrichten und Bilder. Bleibt immer im Client.
`room`	16 Byte	room_id (32 Hex)	Adresse des Chats am Server. Gibt nichts über K preis.
`auth`	32 Byte	Token	Zugriffsberechtigung; der Server speichert nur `sha256(Token)`.
`safety`	8 Byte	12-stelliger Code	Sicherheitscode zum Vergleichen (z. B. am Telefon): stimmt er auf beiden Seiten überein, haben beide denselben Schlüssel.

3 · Verschlüsselung von Nachrichten und Bildern

Jede Nachricht und jedes Bild wird mit **AES-256-GCM** verschlüsselt (authentifizierte Verschlüsselung: Vertraulichkeit *und* Schutz vor Manipulation). Für jeden Vorgang wird ein neuer, zufälliger 12-Byte-Nonce erzeugt.



AAD (mitauthentifiziert, nicht gespeichert): room_id – ein Ciphertext aus einem anderen Chat lässt sich hier nicht entschlüsseln.
Jede Veränderung eines einzigen Bits führt zum Abbruch der Entschlüsselung; der Client zeigt so eine Nachricht nicht an.

Abb. 2 – Aufbau eines verschlüsselten Datensatzes (Nachricht oder Bild-Blob).

Der **Klartext einer Nachricht** ist ein kleines JSON-Dokument; erst dieses wird verschlüsselt. Der Server sieht keinen dieser Inhalte:

Typ	Inhalt (vor der Verschlüsselung)
`text`	Absender-Gerät, Absendername, Sendezeit, Text
`image`	wie oben, plus Bild-Metadaten: `blob_id`, MIME-Typ, Dateiname, Größe, Breite/Höhe
`read`	Lesebestätigung „gelesen bis Nachricht n" – wird gespeichert, aber nicht angezeigt

Bilder werden getrennt als Blob verschlüsselt und hochgeladen; die zugehörige Chat-Nachricht verweist nur über die `blob_id` darauf. Dateiname und Abmessungen stehen ausschließlich in der verschlüsselten Nachricht.

4 · Abläufe

Chat einrichten

- Teilnehmer A wählt „Neu erzeugen": der Client erzeugt K und zeigt Schlüssel, QR-Code und Sicherheitscode.
- A gibt K an B weiter; B wählt „Einfügen". Beide vergleichen den 12-stelligen Sicherheitscode.
- Beim ersten Kontakt mit einer neuen `room_id` legt der Server den Raum nur an, wenn der Request den **Server-Zugangscode** (`X-Invite-Code`) trägt. So kann niemand Fremdes den Server als Speicher missbrauchen. Der zweite Teilnehmer braucht den Code nicht mehr.
- Der Server merkt sich den Raum mit `sha256(Token)`. Ab dann gilt nur noch dieses Token; ein anderes wird mit `401` abgewiesen.

Text senden

- Client baut das Klartext-JSON, verschlüsselt es mit enc_key, sendet `POST /v1/rooms/{room}/messages`.
- Der Server speichert den Ciphertext mit fortlaufender ID und Zeitstempel und schickt ihn per WebSocket an alle verbundenen Clients des Raums.
- Der Empfänger entschlüsselt, prüft den GCM-Tag und zeigt die Nachricht an; später sendet er eine `read`-Bestätigung.

Bild senden

- Bild aus Zwischenablage oder per Drag & Drop (nur PNG/JPG), Prüfung von Format und Pixelzahl.
- Bild wird verschlüsselt und als Blob hochgeladen (`POST …/blobs`, max. 25 MB) → Server liefert `blob_id`.
- Danach geht eine verschlüsselte `image`-Nachricht mit dieser `blob_id` raus.
- Empfänger lädt den Blob, entschlüsselt und prüft ihn und legt ihn – weiterhin verschlüsselt – im lokalen Cache ab.

Verbindung verloren

Nach einem Reconnect holt der Client alles Verpasste über `GET …/messages?after=<letzte ID>`. Nicht zugestellte eigene Nachrichten werden automatisch erneut gesendet, sobald die Verbindung wieder steht.

5 · Server

Was gespeichert wird

Tabelle / Ort	Inhalt
rooms	room_id, sha256(Token), Anlagezeit
messages	ID, room_id, Zeitstempel, Ciphertext
blobs + /data/blobs	blob_id, room_id, Größe, Zeit; die verschlüsselte Datei

Zugriffskontrolle

- Jeder Request trägt `Authorization: Bearer <Token>`; Vergleich in konstanter Zeit.
- Neue Räume nur mit Zugangscode, höchstens 100 Räume.
- Rate-Limit nur für Fehlversuche (10/min pro IP); gültige Tokens werden nie gesperrt.
- Room-IDs erscheinen im Log nur gekürzt.

Härtung und Ressourcen-Limits

Bereich	Maßnahme
Container	distroless-Image ohne Shell, unprivilegierter User 65532, Dateisystem read-only, alle Capabilities entzogen, `no-new-privileges`, 256 MB RAM, 256 Prozesse
Speicher	2 GiB pro Chat; es bleibt immer 1 GiB frei auf der Platte (sonst `507`)
Verbindungen	256 TCP-Verbindungen gesamt, 32 pro IP (vor dem TLS-Handshake); 200 WebSockets gesamt, 8 pro Raum
Größen & Zeiten	Nachricht max. 64 KB, Bild max. 25 MB, Antwort max. 8 MiB; Lese-Timeout 60 s, Bild-Upload max. 10 min
Geheimnisse	Zugangscode nur in `/compose/image-sharing/.env` (Rechte 600), nicht im Image

6 · Client

- **Schlüsselablage:** K und der Zugangscode liegen in der Konfiguration unter `%APPDATA%`, verschlüsselt mit **Windows DPAPI** (an das Windows-Benutzerkonto gebunden). Eine kopierte Konfigurationsdatei ist auf einem anderen Konto wertlos.
- **Bild-Cache:** enthält nur die verschlüsselten Blobs; entschlüsselt wird erst zur Anzeige im Speicher. Ein beschädigter Eintrag wird verworfen und neu geladen.
- **Schlüssel kopieren:** landet nicht im Zwischenablage-Verlauf (Win+V) oder in der Cloud-Zwischenablage und wird nach 60 s wieder gelöscht.
- **Eingaben des Gegenübers sind unvertraut:** Dateinamen werden bereinigt und bekommen die richtige Endung; Bilder über 100 Megapixel werden abgelehnt; Links nur mit http/https; Server-Antworten max. 32 MB.
- **Server-Adresse:** nur `https://` (unverschlüsseltes http nur für localhost-Tests).

7 · Sicherheitsmodell

> **Wenn der Server kompromittiert wird**, erhält ein Angreifer nur Ciphertext. Ohne K kann er weder Texte noch Bilder lesen, und auch keine Nachrichten fälschen, die die Clients akzeptieren: Jede Veränderung scheitert am GCM-Tag. Der gespeicherte Token-Hash erlaubt keinen Rückschluss auf K.

Was der Server (bzw. ein Angreifer dort) sieht

Sichtbar	Nicht sichtbar
• room_id (zufällig, ohne Bezug zu Personen) • Zeitpunkt und Größe jeder Nachricht und jedes Bildes • IP-Adressen der Clients, Online-Zeiten • Anzahl der Nachrichten, Lesebestätigungen als solche nicht unterscheidbar	• Texte, Links, Bilder • Dateinamen, Bildgrößen in Pixeln, MIME-Typ • Namen der Teilnehmer • den Schlüssel K und den AES-Schlüssel

Bewusste Grenzen von Protokoll v1

- **Wiedereinspielen (Replay):** Ein kompromittierter Server könnte eine alte, gültige Nachricht erneut ausliefern. Sie würde korrekt entschlüsselt, ist aber nicht neu. *Abhilfe in v2:* eindeutige Nachrichten-ID im verschlüsselten Klartext.
- **Vertauschte Bilder:** Der Server könnte zu einer Bild-Nachricht einen anderen Blob desselben Chats ausliefern. Fremde Inhalte sind unmöglich, nur Bilder aus diesem Chat. *Abhilfe in v2:* blob_id bzw. Hash des Blobs als AAD binden.
- **Unterdrücken:** Der Server kann Nachrichten zurückhalten oder löschen – Verfügbarkeit ist nicht garantiert, nur Vertraulichkeit und Echtheit.
- **Kein Forward Secrecy:** Es gibt einen statischen Schlüssel pro Chat. Wer K erbeutet (z. B. vom entsperrten Rechner eines Teilnehmers), kann den gesamten Verlauf am Server entschlüsseln. Abhilfe: Chat mit neuem Schlüssel neu anlegen.
- **Metadaten:** Zeitpunkte, Größen und IP-Adressen sind für den Server sichtbar (siehe oben).
- **Endgeräte:** Ein kompromittierter Windows-Rechner eines Teilnehmers ist außerhalb des Schutzbereichs; DPAPI schützt K nur gegenüber anderen Benutzerkonten und kopierten Dateien.

Empfehlungen für den Betrieb

- Schlüssel nur über einen sicheren Kanal weitergeben und den Sicherheitscode immer vergleichen.
- Zugangscode nur an Personen geben, die selbst neue Chats anlegen sollen.
- Bei Verdacht auf Schlüsselverlust: Chat entfernen und mit neuem Schlüssel neu einrichten.
- Privaten TLS-Schlüssel am Host auf Rechte `600` / `640` beschränken (derzeit `644`).

8 · Betrieb

Punkt	Festlegung
Adresse	`https://server.example.com:47000`
Deploy-Verzeichnis	`/compose/image-sharing` – `docker compose up -d --build`
Daten	`./data`: `db.sqlite` (WAL) und `blobs/<room>/<blob_id>` – Backup genügt, alles darin ist verschlüsselt
Zertifikat	`/etc/letsencrypt/live` + `archive` für server.example.com, read-only; Erneuerung ohne Neustart
Prüfung	`GET /healthz` sowie `ImgShare.SelfTest <server> [zugangscode]` (Ende-zu-Ende-Test)
Spezifikation	`docs/PROTOCOL.md` (API, Datenbank, Fehlercodes)

Kryptografische Primitive: AES-256-GCM (96-Bit-Nonce, 128-Bit-Tag), HKDF-SHA256 (RFC 5869), SHA-256, TLS 1.2+ (Go-Standardbibliothek), Windows DPAPI (CryptProtectData, Benutzerbereich). Zufallszahlen: `RandomNumberGenerator` (.NET) bzw. `crypto/rand` (Go).